

Aspect	B-Tree	B+ Tree
Structure	Balanced tree where each node contains keys and pointers	Balanced tree where only leaf nodes store actual data (internal nodes store only keys)
Key Storage	Keys are stored in both internal nodes and leaf nodes	Keys are stored only in the leaf nodes; internal nodes contain only references (keys)
Search Process	Search happens from root to leaf, accessing both internal and leaf nodes	Search follows the same process, but data is only found in the leaf nodes
Insertion	Insert key into appropriate position, node splits if necessary	Insert key into leaf node, split leaf if necessary, middle key pushed up to parent
Deletion	Remove key from the node, may cause underflow and require merging or redistribution	Remove key from leaf, underflow may cause merging or redistribution
Range Queries	Less efficient for range queries (no linked list of leaves)	More efficient due to leaf node linkage (leaves are connected for fast traversal)
Height	May be taller than a B+ Tree (more data stored in internal nodes)	Typically shorter because all data is stored at the leaves
Internal Node Role	Stores both keys and data pointers	Only stores keys to guide the search, no data stored here
Performance	Slower range queries and sequential access	Faster for range queries and sequential access due to linked leaves
Example	Good for balanced indexing of data and used in databases	Ideal for systems requiring efficient range queries, such as filesystems

B+ Tree Insertion & Deletion Steps:

Insertion:

Step 1: Start from the root node and find the correct leaf node where the new key should be inserted.

Step 2: Insert the key into the leaf node while maintaining sorted order.

Step 3: If the leaf node overflows (i.e., the number of keys exceeds the maximum limit), split the node. The middle key moves up to the parent node.

Step 4: If the parent node overflows after receiving the middle key, it too will split, and this process repeats upwards, potentially causing the root node to split and increase the height of the tree.

Deletion:

Step 1: Locate the key in the leaf node.

Step 2: Remove the key from the leaf node.

Step 3: If the leaf node underflows (i.e., has fewer than the minimum allowed keys), attempt to borrow a key from a neighboring sibling. If borrowing is not possible, merge the node with a sibling, and the parent node's corresponding key is deleted.

Step 4: If the parent node underflows after the deletion, it may require further rebalancing or merging.

1. Data Abstraction and Database Systems

a) Define data abstraction. Briefly discuss about different levels of data abstraction.

Data abstraction is the process of hiding complex implementation details and showing only the essential features of the data to the user. It allows users to interact with data at different levels of detail, simplifying the overall system and improving data independence.

Levels of Data Abstraction:

- **Physical Level:** This is the lowest level of data abstraction and describes *how* the data is actually stored on the physical storage media (e.g., hard disk). It deals with low-level details like file organization, indexing, and storage allocation.
- **Conceptual (or Logical) Level:** This level describes *what* data is stored in the database and the relationships among that data. It represents the entire database from the perspective of the database designer and is independent of the physical storage details. It includes entities, attributes, and relationships.
- **View (or External) Level:** This is the highest level of data abstraction and describes only a part of the database relevant to a particular user or application. It hides irrelevant data and provides a customized view of the database, ensuring data security and privacy.
-

b) Illustrate the need for a database system, giving suitable examples.

A database system is needed to manage and organize large amounts of data efficiently and effectively.

Need for a Database System:

- **Data Redundancy Control:** Eliminates or reduces data duplication, saving storage space and ensuring data consistency.
- **Data Consistency:** Enforces rules and constraints to maintain data integrity across the database.
- **Data Sharing:** Allows multiple users and applications to access and share the same data concurrently.
- **Data Security:** Provides mechanisms to control access to sensitive data and protect it from unauthorized use.
- **Data Independence:** Separates data from applications, allowing changes in data storage or organization without affecting applications.
- **Data Backup and Recovery:** Provides tools for creating backups and recovering data in case of system failures.

-

Example:

Consider a university managing student information, course enrollments, and faculty details. Without a database system, this information might be stored in separate files or spreadsheets, leading to:

- **Redundancy:** Student names and addresses might be duplicated in multiple files (e.g., student records, course enrollment lists).
- **Inconsistency:** If a student's address changes, it might be updated in one file but not another, leading to conflicting information.
- **Difficulty in Accessing and Sharing:** Retrieving information across different files would be cumbersome, and sharing data among departments would be challenging.

A database system centralizes all this information, ensuring consistency, reducing redundancy, and facilitating efficient access and sharing for all authorized users.

3. a) Normalization and Different Types of Normal Forms

Normalization is the process of organizing the attributes and tables of a relational database to minimize redundancy and dependency. The goal is to ensure that the database is efficient and avoids anomalies during data insertion, update, or deletion. It involves breaking down a database into smaller, more manageable tables while maintaining relationships between them.

There are several **normal forms (NF)**, each with its own rules to eliminate different types of anomalies:

1. First Normal Form (1NF):

- Ensures that the table contains only atomic (indivisible) values.
- Each record should have a unique identifier (primary key).

- Eliminates repeating groups or arrays in a record.

2. **Second Normal Form (2NF):**

- Achieved when the table is in 1NF, and all non-key attributes are fully functionally dependent on the primary key.
- Removes partial dependencies (i.e., attributes that depend only on part of a composite primary key).

3. **Third Normal Form (3NF):**

- Achieved when the table is in 2NF, and there are no transitive dependencies (i.e., non-key attributes should not depend on other non-key attributes).
- Ensures that each non-key attribute is directly dependent on the primary key.

4. **Boyce-Codd Normal Form (BCNF):**

- A stricter version of 3NF.
- It eliminates any remaining anomalies by ensuring that every determinant in the table is a candidate key.

5. **Fourth Normal Form (4NF):**

- Ensures that multi-valued dependencies (i.e., when one attribute is associated with multiple independent attributes) are removed.
- Tables must be in BCNF before achieving 4NF.

6. **Fifth Normal Form (5NF):**

- Eliminates join dependencies, ensuring that every fact in the table can be reconstructed from the smaller, component facts.
- This form is rarely used in practice.

3. b) Integrity Constraints

Integrity constraints are rules that ensure the accuracy, consistency, and reliability of data within a relational database. They define the valid states and operations allowed on the database. The main types of integrity constraints are:

1. Entity Integrity:

- Ensures that each table has a unique primary key and that no primary key value is **NULL**.
- Guarantees that every row in a table can be uniquely identified.

2. Referential Integrity:

- Ensures that foreign keys in a table correctly reference primary keys in another table.
- Prevents orphan records, i.e., foreign keys that do not exist in the referenced table.

3. Domain Integrity:

- Ensures that the values in each column of a table adhere to predefined data types, formats, and valid ranges.
- For example, a **date** column must contain only valid date values.

4. User-Defined Integrity:

- Defines additional business rules that are not covered by other integrity constraints.
- For example, ensuring that an employee's salary is within a specific range.

4. a) Data Warehousing and Data Mining

Data Warehousing is the process of collecting, storing, and managing large amounts of historical data in a central repository (the data warehouse). This data is used for analysis and decision-making, and it typically integrates data from different sources.

Data Mining is the process of discovering patterns, trends, correlations, and useful information from large datasets using statistical and machine learning techniques.

Data Warehouse Architecture:

A typical data warehouse architecture consists of the following components:

1. Data Sources:

These are the external systems from which data is collected, such as operational databases, flat files, APIs, or online transaction processing (OLTP) systems.

2. ETL Process (Extract, Transform, Load):

- **Extract:** Data is extracted from the source systems.
- **Transform:** Data is cleaned, formatted, and transformed into a suitable format for the data warehouse.
- **Load:** The transformed data is loaded into the data warehouse.

3. Data Warehouse:

- A central repository that stores data in a format optimized for querying and analysis (usually in the form of star or snowflake schemas).

4. Data Marts:

- Subsets of the data warehouse that are focused on specific business areas (e.g., sales, finance).

5. OLAP (Online Analytical Processing):

- Tools used to perform complex queries and analysis on the data warehouse, often with multidimensional views.

6. Data Mining Tools:

- Tools that use advanced analytics, machine learning, and statistics to extract patterns and trends from the data.

7. Presentation Layer:

- The interface through which end-users access and interact with the data warehouse using reports, dashboards, and visualization tools.
-

4. b) Deadlock Detection and Recovery

Deadlock Detection refers to the process of identifying situations in which two or more transactions are waiting for each other to release resources, causing a cycle of dependencies that prevent any of the transactions from proceeding.

Once deadlock is detected, the system must recover from it. The steps involved in deadlock recovery are:

1. Identify the Deadlocked Transactions:

- A detection algorithm checks for cycles in the wait-for graph, which represents transactions and their dependencies on resources. Transactions in the cycle are deadlocked.

2. Select a Victim:

- One or more transactions involved in the deadlock must be aborted. The system chooses a "victim" transaction, typically based on factors such as the transaction's age, the amount of work done, or the resources it holds.

3. Rollback the Victim Transaction:

- The victim transaction is rolled back, releasing any locks or resources it holds. This action breaks the deadlock cycle.

4. Re-initiate or Retry the Transactions:

- After rollback, the system may either attempt to restart the victim transaction or allow other transactions to proceed.

5. Prevention of Future Deadlocks:

- The system can also implement prevention techniques, such as **timeout mechanisms** or **resource allocation policies**, to minimize the chances of future deadlocks.

a) Why is B+ tree efficient than B tree? Explain how data retrieval, insertion and deletion are done using B tree and B+ tree indices.

B+ trees are generally more efficient than B trees for disk-based databases due to their structure, which optimizes disk I/O operations.

- **Efficiency of B+ Tree over B Tree:**

- **All data in leaves:** In a B+ tree, all data pointers reside in the leaf nodes, forming a sorted linked list. This allows for efficient range queries and sequential access, as all data records are at the same level. In contrast, B trees can have data pointers at any level, making range queries less efficient as they may require traversing multiple levels.
- **Fixed-size internal nodes:** Internal nodes in B+ trees only store keys and child pointers, leading to fixed-size nodes that can hold more keys. This reduces the height of the tree and thus the number of disk I/O operations required to reach a leaf node. B trees, with data pointers in internal nodes, can have variable-sized nodes.

- **Data Retrieval:**

- **B Tree:** To retrieve data, traverse the tree from the root, comparing the search key with keys in each node. If a matching key is found in an internal node, the corresponding data pointer is followed. If not, the search continues down to the appropriate child node until the data is found or it's determined not to exist.
- **B+ Tree:** To retrieve data, traverse the tree from the root, comparing the search key with keys in internal nodes to navigate to the correct leaf node. Once the leaf node is reached, the data record is accessed directly using the data pointer associated with the key. For range queries, after finding the starting key in a leaf node, the linked list of leaf nodes can be traversed sequentially.

- **Insertion:**

- **B Tree:** Find the appropriate node for insertion. If the node has space, insert the key and data pointer. If the node is full, split it into two,

promoting the middle key to the parent node. This splitting can propagate up the tree.

- **B+ Tree:** Find the appropriate leaf node for insertion. If the leaf node has space, insert the key and data pointer. If the leaf node is full, split it into two, copying the middle key to the parent node and linking the new leaf node. This ensures all data remains in the leaf level.
- **Deletion:**
 - **B Tree:** Locate the key to be deleted. If it's in a leaf node, remove it. If it's in an internal node, replace it with its in-order successor or predecessor from a child node, and then delete that successor/predecessor from its original location. If a node becomes underflowed after deletion, it might need to merge with a sibling or redistribute keys.
 - **B+ Tree:** Locate the key in the leaf node and remove it. If the leaf node becomes underflowed, it might need to merge with a sibling or redistribute keys. If a key is removed from an internal node (because its corresponding data entry in a leaf node was deleted), the key in the internal node might need to be updated or removed if it no longer serves as a separator.

a) Simple Locking Protocol and its Problems

A simple locking protocol, often used in database concurrency control, ensures that transactions acquire a lock on a data item before accessing it and release the lock after completing the access. This prevents concurrent transactions from interfering with each other's operations.

Problems that may arise while using simple locking:

- **Deadlock:** This occurs when two or more transactions are waiting indefinitely for each other to release a resource (lock) that they need. For example, Transaction

A holds a lock on item X and needs a lock on item Y, while Transaction B holds a lock on item Y and needs a lock on item X.

- **Starvation:** A transaction may repeatedly lose the race for a lock and never get to execute if other transactions consistently acquire the lock before it.
 - **Reduced Concurrency:** Simple locking can limit the degree of concurrency, as transactions might have to wait for locks even if their operations are on different parts of the data that wouldn't cause a conflict.
 -
-

b) Transitive Functional Dependencies and Normalization

Transitive Functional Dependency

A transitive functional dependency occurs when a non-key attribute is dependent on another non-key attribute, which in turn is dependent on the primary key. In simpler terms, if $A \rightarrow B$ and $B \rightarrow C$, then a transitive dependency exists if $A \rightarrow C$ (where B is not a superkey).

Merits and Demerits of Normalization

Merits of Normalization:

- **Reduced Data Redundancy:** Eliminates or significantly reduces duplicate data, saving storage space and preventing inconsistencies.
- **Improved Data Integrity:** Ensures data consistency and accuracy by minimizing update, insertion, and deletion anomalies.
- **Better Data Organization:** Structures the database logically, making it easier to understand and manage.
- **Enhanced Query Performance:** In some cases, normalized data can lead to more efficient data retrieval by reducing the amount of data that needs to be scanned.
-

Demerits of Normalization:

- **Increased Query Complexity:** Queries might become more complex as they often require joining multiple tables to retrieve complete information.
- **Potential Performance Overhead:** Joining multiple tables can sometimes lead to performance overhead, especially in systems with very high transaction volumes.
- **More Tables:** Normalization typically results in a larger number of tables, which can increase the complexity of database design and management.
- **Difficulty in Understanding:** Highly normalized schemas can sometimes be harder for users to understand due to the fragmentation of data across many tables.

7. Customer table:

Customer_ID	Name	City
1	Mike	New York
2	Alice	Los Angeles
3	Sara	Chicago
4	Jane	New York

Orders Table:

Order_ID	Customer_ID	Order Date	Order total
100	1	2023-07-01	100.00
101	2	2023-06-15	50.00
102	3	2023-07-05	150.00
103	1	2023-07-07	75.00
104	4	2023-07-02	200.00

Write the SQL statements for the following:

- Find the total number of orders placed by each customer, excluding orders placed in June.
- Find the customer who has placed the highest total order value.
- List all orders placed on specific dates (eg., 2023-07-04 and 2023-07-06) and their corresponding customer names.
- Find the average order value for each city.
- Identify customers who haven't placed any orders.

5*2

7. SQL Statements for Customer and Orders Tables

i. Find the total number of orders placed by each customer, excluding orders placed in June.

Code



```
SELECT c.Name, COUNT(o.OrderID) AS TotalOrders
FROM Customer c
JOIN Orders o ON c.Customer_ID = o.Customer_ID
WHERE STRFTIME('%Y-%m', o.Order_Date) != '2023-06'
GROUP BY c.Name;
```

ii. Find the customer who has placed the highest total order value.

Code



```
SELECT c.Name, SUM(o.Order_total) AS TotalOrderValue
FROM Customer c
JOIN Orders o ON c.Customer_ID = o.Customer_ID
GROUP BY c.Name
ORDER BY TotalOrderValue DESC
LIMIT 1;
```

iii. List all orders placed on specific dates (e.g., 2023-07-04 and 2023-07-06) and their corresponding customer names.

Code



```
SELECT o.OrderID, c.Name, o.Order_Date, o.Order_total
FROM Orders o
JOIN Customer c ON o.Customer_ID = c.Customer_ID
WHERE o.Order_Date IN ('2023-07-04', '2023-07-06');
```

iv. Find the average order value for each city.

Code



```
SELECT c.City, AVG(o.Order_total) AS AverageOrderValue
FROM Customer c
JOIN Orders o ON c.Customer_ID = o.Customer_ID
GROUP BY c.City;
```

v. Identify customers who haven't placed any orders.

Code



```
SELECT c.Name
FROM Customer c
LEFT JOIN Orders o ON c.Customer_ID = o.Customer_ID
WHERE o.OrderID IS NULL;
```

a) Types of Database-System Architecture

Database systems can be structured in different architectures depending on how the components (such as databases, application servers, and user interfaces) are organized and interact with each other. The primary types of database-system architectures are:

1. Centralized Database Architecture:

- **Description:** In a centralized database architecture, the entire database is stored on a single server or a central machine. All database-related operations, including query processing, are handled by this single server.
- **Advantages:** Simplified management, fewer points of failure.
- **Disadvantages:** Single point of failure, limited scalability, performance bottlenecks with heavy load.

2. Client-Server Database Architecture:

- **Description:** This is a more distributed approach where the database is stored on a server (the database server), and clients (user applications or machines) connect to it for data retrieval and modification. The server handles database management, while the client sends requests for data.
- **Two-tier Architecture:**
 - Involves direct communication between client and server.
 - Clients send queries, and the server processes and returns the data.
- **Three-tier Architecture:**
 - Involves an intermediary layer between the client and server, typically an application server that handles business logic.
 - Client → Application Server → Database Server
- **Advantages:** Scalability, improved performance, better fault tolerance.
- **Disadvantages:** Complexity in management, network dependency.

3. Distributed Database Architecture:

- **Description:** In a distributed database system, the database is spread across multiple physical locations. The system is composed of several interconnected databases that work together, appearing as a single logical database to the user.
- **Types of Distributed Database Architecture:**
 - **Homogeneous:** All databases run the same database management system (DBMS) and share the same schema.
 - **Heterogeneous:** Different DBMS types are used in different locations, which may require more complex data integration.
- **Advantages:** Improved performance and reliability, fault tolerance, and scalability.
- **Disadvantages:** Complexity in synchronization and management, potential network issues.

4. Cloud Database Architecture:

- **Description:** Cloud databases are hosted and managed on cloud platforms. They can be centralized or distributed but benefit from the scalability and flexibility of cloud infrastructure.
 - **Advantages:** Scalability, pay-per-use pricing, high availability.
 - **Disadvantages:** Security concerns, reliance on cloud service providers.
-

b) Distributed System and Distributed Database System

Distributed System

A **distributed system** is a system that consists of multiple independent components (such as computers, nodes, or machines) that communicate and coordinate with each other to achieve a common goal. These components may be located in different physical locations but work together as a single system. A distributed system is characterized by:

- **Concurrency:** Multiple components can execute tasks simultaneously.
- **Scalability:** It can scale to handle increased load by adding more components.
- **Fault Tolerance:** If one component fails, others can continue to function, ensuring system reliability.
- **Transparency:** Users and applications interact with the system as though it is a single entity, even though it is distributed.

Examples of Distributed Systems:

- Cloud computing platforms
 - Peer-to-peer (P2P) networks
 - Internet of Things (IoT) networks
 - Microservices architecture
-

Distributed Database System

A **distributed database system (DDBMS)** is a database system that manages data across multiple physical locations or servers while maintaining the illusion of a single, unified database. It is a type of distributed system designed specifically to store, manage, and process data across multiple locations. The data may be stored in various nodes of a network, and the system provides tools for accessing, updating, and maintaining data in a coordinated manner.

Key Components of Distributed Database Systems:

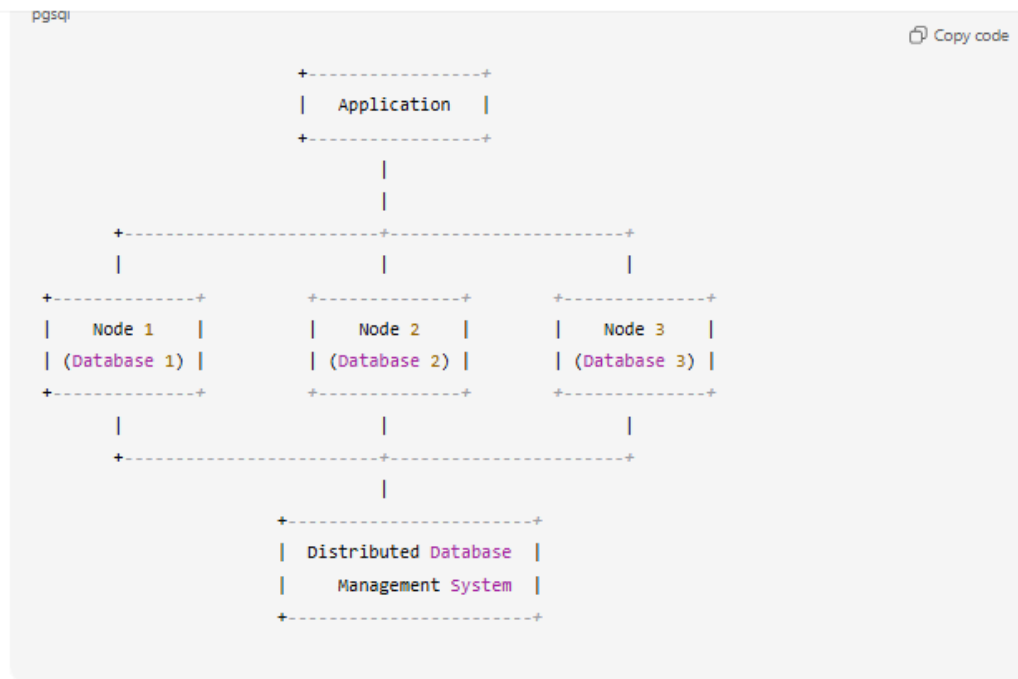
1. **Data Distribution:** The database is distributed across multiple nodes (physical locations or servers).
 - **Horizontal Partitioning:** Data is divided into rows and distributed across different nodes.
 - **Vertical Partitioning:** Data is divided by columns and distributed across nodes.
 - **Replication:** The same data is stored at multiple locations to ensure availability and fault tolerance.
 2. **Distributed Query Processing:** The system must handle queries that involve accessing data from multiple locations and combine the results seamlessly.
 3. **Synchronization:** The system ensures that data is consistent across all locations by using methods like two-phase commit or timestamp ordering to maintain consistency.
 4. **Concurrency Control:** It ensures that multiple transactions can be executed simultaneously without causing conflicts or anomalies.
 5. **Fault Tolerance:** Distributed databases have mechanisms in place (such as replication) to ensure the system continues to function even if one or more nodes fail.
-

Figure: Distributed System and Distributed Database System

The figure below provides an illustrative example of both concepts:

1. **Distributed System:**
 - In a distributed system, multiple independent machines (or nodes) are connected through a network, and each machine executes tasks as part of a larger system.
2. **Distributed Database System:**

- In a distributed database system, the data is partitioned and replicated across multiple nodes (databases). Each node may perform query processing locally, but they communicate with other nodes to retrieve and process data from remote locations.



- **Nodes:** Represent the physical locations where data is stored. Data can be replicated or partitioned across multiple nodes.
- **Database Management System (DBMS):** Manages the distribution, query processing, and consistency of data across all nodes.

🔍 🔍 🔍 🔍 🔍 🔍

